MycoAI Retrieval

Fungal Species Identification via Image Retrieval

Graduation Thesis

MycoAI Lab – HUST & Utrecht

June 2025

Outline

Why Fungi?

The question:

> *"What can you do to help a mycologist?"*

The bottleneck: Manual lookup. Mycologists spend hours comparing colony photos against reference books.

Goal: Build a tool mycologists *actually use* – not chase benchmarks.

Advisors

Dr. Oanh Nguyen (HUST) – Computer Vision

Prof. Duong Vu (Utrecht) – Mycology Domain Expertise

Approach

Research ∩ Engineering. CV techniques applied thoughtfully to a domain that needs them.

The Identification Challenge

Manual identification is:

- Time-consuming (expert hours per sample)
- Subjective (inter-observer variability)
- Not scalable (expanding taxonomy)

MycoAI Retrieval – image-based retrieval system: Query → features → nearest neighbors → species prediction.

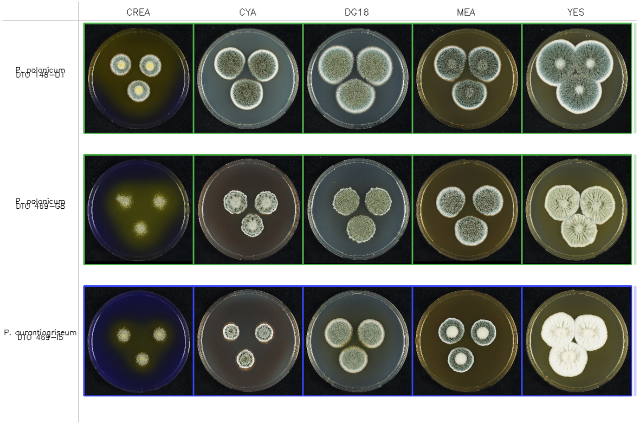
- Open-set: new species via DB insert, no retraining
- Interpretable: shows similar reference images

Dataset

435 Petri dish images
8 *Penicillium* species
31 strains
7 growth media
2 angles (oblique / reverse)
3 colonies/dish → **1,305** segments

Species → Strain → Medium → Angle → Segment

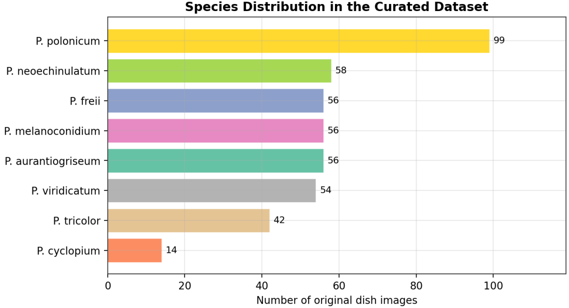
- **8 Species**: *P. polonicum, freii, neoechinulatum, aurantiogriseum, viridicatum, tricolor, melanoconidium, cyclopium*
- **7 Media**: CREA, CYA, CYA30, CYAS, DG18, MEA, YES
- **Oblique**: surface morphology, texture
- **Reverse**: agar pigment diffusion
- Within-species variation across strains and media



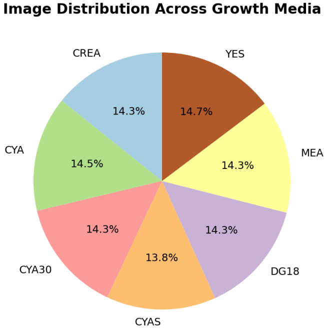
3 strains across 5 media. Green = same species, Red = different.

Dataset Composition

Species Distribution



Class-imbalanced: 14–99 images/species



Media Coverage

7 media, 60–64 images each – balanced

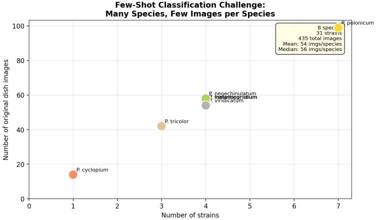
Constraints:

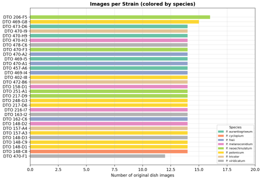
- 8 species, only 24 training strains
- Per-strain: 14 images (7 media $\times$ 2 angles)
- Mean 54 images/species, median 56

Strain-level split:

- Test: 7 held-out strains (unseen isolates)
- Harder than random image split
- Reflects real deployment

$\rightarrow$ Traditional CNN classifier would overfit

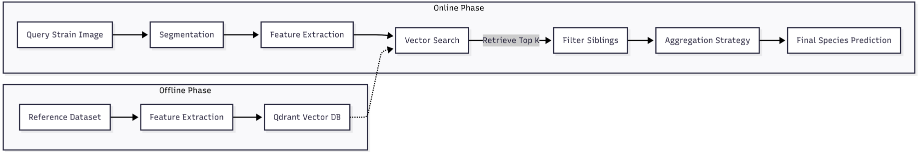




Scope – Three Domains

1. **Algorithmic Research**
   - Colony segmentation (K-Means, contour, YOLO)
   - Feature extraction (hand-crafted + deep learning)
   - Vector indexing, similarity search, weighted aggregation
2. **System Engineering**
   - Production web app: React + FastAPI + PostgreSQL + Qdrant
   - RBAC (User / Data Owner), species catalog, feedback workflow
3. **Process Innovation**
   - 5-agent autonomous experiment system (Autolab)

Retrieval Pipeline Architecture



Indexing Phase (offline): Segment → Extract features → Store in Qdrant

Query Phase (online): Upload → Segment → Embed → KNN → Aggregate → Predict

Why Retrieval over Classification?

Open-set

New species = DB insert only. No architecture change, no retraining.

Interpretable

Shows mycologist the actual similar reference images. Expert can visually validate.

Data efficient

No need to learn all decision boundaries. Relies on feature quality + nearest-neighbor.

Segment → Extract → Search → Aggreg

Modular design:

- Swap extractors independently
- Swap segmentation freely
- Experiment-friendly for A/B testing

Pipeline – Four Stages

1. **Preprocessing & Segmentation**
   - Hough Circle → background mask → K-Means colony extraction
   - Local K=2 Shrink for halo removal (MEA, YES)
   - Alternatives: Contour (Canny), YOLOv8n-seg
2. **Feature Extraction**
   - Hand-crafted: HOG (3,780d), Gabor (40d), ColorHistHS (64d)
   - Deep: ResNet50, MobileNetV2, EfficientNetB1 – fine-tuned
3. **Vector Database (Qdrant)**
   - Cosine similarity, 13 named vectors per point, sibling filtering
4. **Aggregation**
   - Weighted sum of per-segment neighbor similarities, normalized by frequency

Colony Segmentation

Method	Colonies	Flare-robust?	Labels?
K-Means (K=2, HSV)	3	Partial (Local K=2)	No
Contour (Canny + Circ.)	2–3	Yes (edge-based)	No
YOLOv8n-seg	Variable	Yes (learned)	Yes

Local K=2 Shrink (key innovation):

- Bright colonies on MEA/YES create light halos
- Second K=2 on value channel within each candidate region
- Morphological erosion strips borderline halo pixels

Production choice: K-Means + Local K=2. YOLO reserved for cross-validation.

Feature Extractors Comparison

Extractor	Dim	Accuracy	Δ Pretrained
EfficientNetB1 (FT)	1,280	**83.3%**	**+20.3%**
ResNet50 (FT)	2,048	78.6%	+15.6%
MobileNetV2 (FT)	1,280	78.6%	+18.6%
HS+ResNet50 (Hybrid)	2,112	72%	–
ColorHistHS	64	65%	–
ResNet50 (Pretrained)	2,048	63%	baseline
Triplet Loss (EffNetB1)	128	64.3%	–
Gabor	40	55%	–
HOG	3,780	52%	–

Key: Fine-tuning = +15–20%. Triplet loss fails (19% worse) on small datasets.

Fine-Tuning & Training

Setup:

- 1,011 training images (24 strains)
- 294 validation images (7 held-out strains)
- Augmentation: HFlip(0.5), Rot$\pm 10^\circ$
- CrossEntropyLoss, Adam, lr=0.0001
- 50 epochs max, early stopping patience=10

Why not triplet loss?

- Needs 10$\times$ more data than classification
- Reduced dim (128 vs 1280) limits capacity
- No hard negative mining
- Class imbalance biases triplet sampling

Model	**Acc.**	**Params**
EfficientNetB1	83.3%	7.8M
ResNet50	78.6%	25.6M
MobileNetV2	78.6%	3.5M

MobileNetV2

78.6% at 7$\times$ fewer params. Edge-deployable.

Vector Database & Aggregation

Qdrant:

- 1,305 colony segments indexed
- 13 named vectors per point
- Dynamic filtering (sibling exclusion)
- Environment filter: same-media / all-media

Environment Strategies:

- E1 (All): train on all media → **best**
- E2 (Balanced): equal per-media samples
- E3 (Single-Env): test one medium
- E4 (Leave-One-Out): exclude one medium

Weighted Aggregation:

$$S_c = \frac{\sum s_{i,j} \cdot \mathbb{1}[c_{i,j} = c]}{\sum \mathbb{1}[c_{i,j} = c]}$$

- Normalizes by frequency
- Typical: 18 segments per strain
- Per-segment KNN → merged species score

Optimal k**: 7** (cross-validated)

Autoresearch – Staircase Chart

Hundreds of combinations tested: distance metrics, aggregation, k-values, media filters

- Each dot = one experiment (formula $\times$ algorithm pair)
- Green dots = new best F1 $\rightarrow$ staircase steps up
- Gray dots = below running best
- Labels = strategy name on each green dot

Staircase chart from `results/autoresearch/retrieval.csv`

Overall Performance – EfficientNetB1

Species	Acc.
P. polonicum	100%
P. freii	100%
P. neoechinulatum	100%
P. aurantiogriseum	83%
P. viridicatum	83%
P. tricolor	67%
P. melanoconidium	50%
Overall	**83.3%**

Key findings:

- **Fine-tuning critical:** +15–20%
- **MobileNetV2:** 78.6% at 3.5M params
- **Hand-crafted plateau:** max 65%
- **Triplet loss:** 19% worse

Confusions:

- *P. melanoconidium* ↔ *P. aurantiogriseum* (dark colonies)
- *P. tricolor* ↔ *P. viridicatum* (green pigment)

5-Fold Cross-Validation

Env	Agg	K	Mean	Std	Max
E1	weighted	7	0.833	0.052	0.905
E1	weighted	9	0.829	0.048	0.905
E1	uni	5	0.810	0.065	0.905
E2	weighted	9	0.805	0.072	0.905

Conclusions:

- **E1 (all environments)** beats E2 – diversity helps
- **Weighted aggregation** outperforms uniform
- $k = 7$ optimal: smaller noisy, larger dissimilar neighbors
- **Stable:** std 5–7% across folds

100 total runs: 5 folds $\times$ 2 env $\times$ 2 agg $\times$ 5 K values

Feature Space & Confusion Analysis

t-SNE (ResNet50 fine-tuned):

- Clear clusters: *P. polonicum, freii, neoechinulatum*
- Overlap: *P. melanoconidium / P. aurantiogriseum*
- **No environment clustering!**

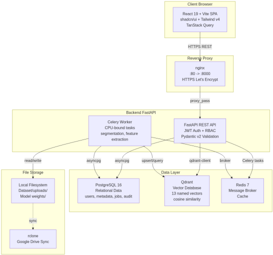
→ Model learns **environment-invariant, species-discriminative** features

Bottleneck species:

- 3 species at **100%** (perfect)
- 2 at **83%**
- *P. tricolor*: 67%
- *P. melanoconidium*: 50% – hardest

Dark-colony species visually indistinguishable even for experts. Hard negative mining or two-stage classifier could help.

System Architecture



6 services:

- React 19 SPA (Vite + TypeScript)
- FastAPI (Python 3.13)
- PostgreSQL (governance)
- Qdrant (vector search)
- Redis + Celery (async)

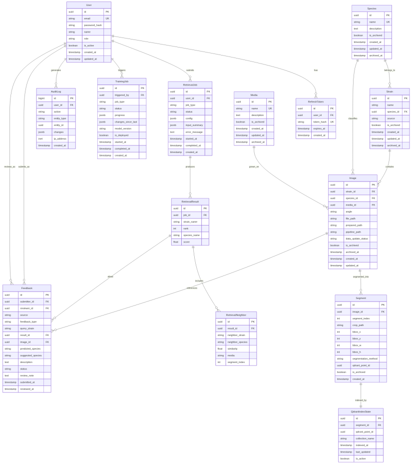
Dual-DB strategy:

- PostgreSQL: ACID, RBAC, audit
- Qdrant: 1,280-dim embeddings
- Bridge table prevents drift

Technology Stack

Layer	Tech	Why
Frontend	React 19 + Vite + TS	SPA, HMR, type safety
UI	shadcn/ui + Tailwind v4	Accessible, utility-first
Data	TanStack Query	Caching, auto-refetch
Backend	FastAPI (async)	Auto-docs, Pydantic v2
ORM	SQLAlchemy 2.0 async	DeclarativeBase
Vector DB	Qdrant	Cosine KNN, named vectors
Tasks	Celery + Redis	Retries, monitoring
Auth	python-jose + bcrypt	JWT access/refresh
CV	OpenCV + scikit-learn	KMeans, contour
Deploy	Docker Compose	Single-VM reproducible
CI/CD	GitHub Actions	Lint, typecheck, test

Database Design



14 tables:

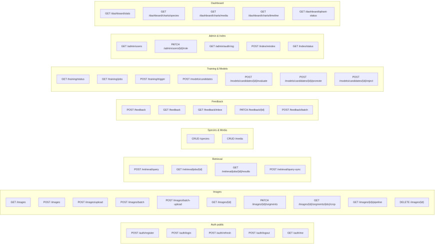
- Core: Users, Species, Media, Strains, Images, Segments
- Jobs: RetrievalJob, RetrievalResult, RetrievalNeighbor
- Governance: Feedback, AuditLog, TrainingJob
- State: QdrantIndexState, SystemState

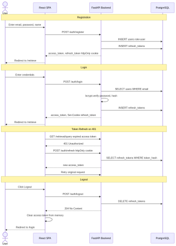
Key patterns:

- UUIDv4 PKs – distributed-safe
- Soft delete (`is_archived`) – no permanent deletion
- JSONB for config, progress, diffs
- `data_update_status` drives Qdrant re-index

REST API – `/api/v1/`:

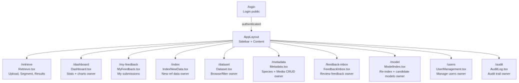
- 47 endpoints, 9 resource groups
- RFC 7807 error format
- Async job: 202 Accepted + polling





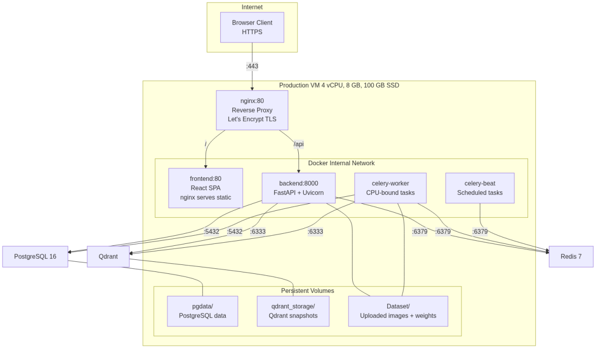
- Access token: 1h, HS256
- Refresh token: 30d, httpOnly cookie
- RBAC: User / Data Owner roles

Frontend Application



11 pages, role-gated:

- **Retrieve** – 3-step upload/review/results
- **Dataset** – filter, group, archive
- **Dashboard** – SVG charts, index status
- **Model Index** – re-index, candidate models
- **Feedback** – review user contributions
- **Metadata** / **Users** / **Audit** – owner-only



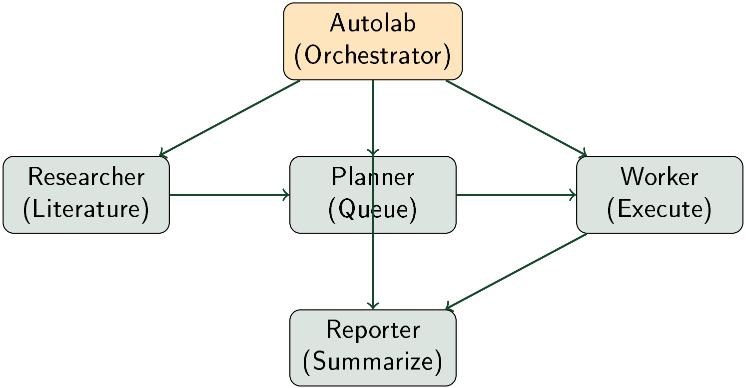
Docker Compose:

- 6 services, internal network
- Persistent volumes (DB, uploads, Qdrant)
- Nginx reverse proxy

CI/CD (GitHub Actions):

- Backend: ruff → mypy → pytest
- Frontend: ESLint → tsc → build
- Manual deploy after PR merge

Autolab – 5-Agent Orchestration



Worker isolation: git worktrees under `.runtime/worktrees/`. Max 2 concurrent. Removed after run, best retained until merged.

Agent Roles

Agent	Model	Responsibility
Autolab	BigBrain	Orchestrator – delegates to all subagents
Researcher	BigBrain	Literature scout: web/PDF → hypotheses
Planner	MidBrain	Queue coordinator: assigns run_ids
Worker	MiniBrain	Isolated runner: worktree, execute, log
Reporter	MiniBrain	Status summarizer: CSV → F1 summary

Hypothesis pipeline: Researcher proposes → Planner validates duplicates → Worker tests in isolation → Reporter logs best F1

Guardrails & Context Management

Reducing Hallucinations:

1. **SDD (Spec-Driven)**
   - Living Spec, speckit toolchain
   - Drift detection and resolution
2. **TDD (Test-Driven)**
   - Failing test $\rightarrow$ fix $\rightarrow$ verify
   - Every change has evidence
3. **Specialized Skills**
   - Bounded instructions (react-best-practices, fastapi)

Context Management:

1. **Caveman Mode**
   - $\sim$75% token reduction
   - Technical substance intact
2. **Subagents + Worktrees**
   - Isolated per task
   - Only relevant context loaded
3. **Reusable Skills**
   - Encode workflows as skills
   - Constrain to validated patterns

Summary & Future Work

Achievements:

- Retrieval: **83.3%** on 8 *Penicillium* species
- Web app: 11 pages, 47 endpoints, 14 tables
- 5-agent autonomous experiment system
- Open-set, modular, interpretable
- 13 feature extractors, 3 segmentation methods

Future:

- Interactive seg $\rightarrow$ auto-finetune YOLO
- One-click in-app model training
- Batch processing, plate-level analysis
- Medium-aware ensemble weighting
- Integration with MycoBank / Index Fungorum

Core Philosophy

AI not only as the product – but as the primary driver of the development process.

Thank You

Questions?

MycoAI Retrieval – Graduation Thesis, June 2025

Advisors: Dr. Oanh Nguyen (HUST) • Prof. Duong Vu (Utrecht)